Лекция 10. Наблюдаемость ML-сервиса

Курс: «Управление процессами машинного обучения MLOps» Продолжительность: 2 академических часа Связанная работа: ЛР11 «Prometheus, Grafana, структурированные логи и alert» Актуализация: 20 октября 2026 года

Цель лекции

Научить проектировать наблюдаемость как проверяемый контракт эксплуатации, а не как набор красивых графиков. После занятия студент должен различать telemetry и evidence, выбирать ограниченные labels, вычислять request rate, error ratio и latency quantiles, связывать запрос с версией модели, формулировать SLI/SLO, создавать alert с окном и runbook и объяснять границу между мониторингом сервиса и мониторингом качества модели.

План занятия

Время	Содержание	Действие преподавателя
0–10 мин	Наблюдаемость и диагностика	Сравнить dashboard и ответ на вопрос «почему отказ?»
10–23 мин	Метрики, логи, traces	Разделить сигналы и их роли
23–37 мин	RED и ML identity	Спроектировать rate, errors, duration и model info
37–50 мин	Prometheus и labels	Показать route template и cardinality explosion
50–62 мин	PromQL и histogram	Вычислить rate, 5xx ratio и p95
62–72 мин	SLI, SLO и error budget	Перевести ожидание в окно и долю хороших событий
72–82 мин	Alert и runbook	Разобрать pending, firing, resolved и действие
82–90 мин	ЛР11	Назначить нагрузку и индивидуальные инциденты

1. Наблюдаемость — способность задавать новые вопросы

Мониторинг обычно отвечает на заранее выбранные вопросы: доступен ли сервис, сколько запросов, превышен ли порог. Наблюдаемость шире: по внешним сигналам инженер должен объяснить внутреннее состояние и исследовать ситуацию, которую не описывал заранее.

Начните с вопроса: «После выпуска версии 7 выросли ошибки только у `/predict`; какие запросы, экземпляры и model version затронуты?» Один график CPU не отвечает. Нужны метрики для масштаба, структурированные логи для конкретного события и, в распределённой системе, traces для пути запроса.

2. Telemetry не равно evidence

Telemetry — поток измерений. Evidence — сохранённое доказательство конкретного вывода: PromQL-запрос с окном, снимок значений, request ID, JSON-log, версия dashboard и связанный release manifest. Скриншот без времени, запроса и версии конфигурации слаб.

В ЛР11 evidence создаётся воспроизводимой нагрузкой и JSON-report. Каждый студент должен связать свой запрос с логом и resolved model version.

3. Три основных сигнала

Сигнал	Сильная сторона	Ограничение
Metrics	дешёвая агрегация и alerting	теряют детали события
Logs	контекст конкретного события	поиск и хранение дороже
Traces	путь запроса через компоненты	требуют propagation и sampling

В обязательной части курса реализуются metrics и JSON logs. Traces рассматриваются как расширение: request ID не является trace ID, но уже создаёт учебную корреляцию.

4. RED для HTTP-сервиса

RED — Rate, Errors, Duration:

1. Rate: запросов в секунду по route template.
2. Errors: доля 5xx и отдельно наблюдаемые 4xx.
3. Duration: распределение времени ответа, а не только среднее.

4xx не всегда ошибка сервиса: 422 может означать корректно отклонённый невалидный запрос. Однако рост 422 может указывать на несовместимый клиентский контракт. Поэтому dashboard показывает статусы, а SLO явно определяет, какие события считаются плохими.

5. ML identity как эксплуатационный сигнал

Одинаковый endpoint может обслуживаться разными версиями модели. Лог и метрика `model_info` должны сообщать `model_name`, `model_version` и `source_commit`. Это связывает ЛР11 с release manifest ЛР10.

Не добавляйте model version к каждой высокочастотной метрике без оценки cardinality. Для учебного сервиса одна активная version допустима; при большом числе моделей identity лучше хранить в отдельной info metric и логах.

6. Метрики Prometheus

Четыре типа:

- Counter только растёт, кроме reset при рестарте процесса.
- Gauge может расти и уменьшаться.
- Histogram распределяет наблюдения по bucket и создаёт `_bucket`, `_sum`, `_count`.
- Summary вычисляет quantiles на клиенте и хуже агрегируется между экземплярами.

Для запросов используйте counter и histogram. `rate()` применяют к counter до агрегации, чтобы корректно учитывать resets.

7. Имена и единицы

Имя должно содержать предмет, единицу и тип, когда это принято: `http_request_duration_seconds`, а не `latency_ms_value`. Базовые единицы Prometheus — seconds, bytes и безразмерные ratios.

Описание метрики отвечает на вопрос, что измеряется. Метрика не должна менять смысл между версиями, сохраняя имя.

8. Labels и cardinality

Каждая уникальная комбинация labels — отдельный time series. Безопасные labels имеют ограниченное множество: `method`, route template, status class, controlled model version. Опасные labels: `request_id`, user ID, полный URL, timestamp, текст ошибки, prediction probability.

Сравните:

```
route="/predict"                # ограниченно
path="/users/846291/predict"   # потенциально безгранично
```

Request ID помещается в JSON-log, а не в metric label. Официальная практика Prometheus рекомендует расследовать dimensions с потенциальной cardinality порядка сотен и выше.

9. Middleware как граница измерения

HTTP middleware видит метод, route, статус и полную продолжительность. Он должен измерять и успешные, и отклонённые запросы. Route template обычно становится известен после маршрутизации, поэтому label читается из resolved route, а неизвестные пути объединяются в `unmatched`.

Нельзя использовать сырой `request.url.path`: `/items/1`, `/items/2` создадут разные series.

10. PromQL: request rate

```
sum by (route) (rate(http_requests_total[5m]))
```

Сначала `rate`, затем `sum`. Окно 5 минут сглаживает шум, но медленнее реагирует. На малом учебном потоке короткие окна могут возвращать нестабильные значения; генератор нагрузки делает наблюдение воспроизводимым.

11. PromQL: error ratio

```
sum(rate(http_requests_total{status=~"5.."}[5m]))
/
clamp_min(sum(rate(http_requests_total[5m])), 0.001)
```

Знаменатель защищён от деления на ноль. Для production лучше отдельно обработать отсутствие трафика: нулевая error ratio при отсутствии запросов не доказывает здоровье сервиса.

12. Histogram и p95

```
histogram_quantile(
  0.95,
  sum by (route, le) (rate(http_request_duration_seconds_bucket[5m]))
)
```

Классические histogram buckets cumulative; label `le` обязателен при агрегации. p95 — оценка внутри bucket. Она не означает, что каждый запрос быстрее порога, и зависит от выбранных границ bucket.

Среднее скрывает хвост. p95 без request rate также вводит в заблуждение: один медленный запрос в редком потоке и массовая деградация — разные ситуации.

13. Structured JSON logs

Минимальное событие запроса:

```
{
  "event": "http_request",
  "request_id": "lr11-student-01-0007",
  "method": "POST",
  "route": "/predict",
  "status": 200,
  "duration_ms": 12.4,
  "model_name": "flight-delay-classifier",
  "model_version": "3"
}
```

Не логируйте признаки, токены, персональные данные, полный request body и prediction probability по умолчанию. Structured означает стабильные ключи и типы, а не строку, похожую на JSON.

14. Correlation

Клиент может передать безопасный `X-Request-ID`; сервис ограничивает длину и генерирует UUID при отсутствии. ID возвращается в response header и body ошибки. По нему студент связывает HTTP-response и JSON-log.

Доверять произвольному ID как секрету нельзя. Его используют только для корреляции, экранируют при выводе и не помещают в labels.

15. SLI и SLO

SLI — измеряемый показатель. SLO — целевое значение SLI за окно.

Пример учебного SLO:

```
За 15 минут не менее 99% валидных запросов /predict завершаются без 5xx,
а p95 latency не превышает 500 ms при не менее 50 запросах.
```

Определение включает population, хорошие события, порог, окно и минимум трафика. «Сервис должен быть быстрым» не является SLO.

16. Error budget

Для availability SLO 99% допустима 1% плохих событий в выбранном окне. Error budget помогает обсуждать риск изменений, но на короткой ЛР статистика мала. Используйте его как модель решения, не имитируйте production SLA.

Покажите, что 1 ошибка из 20 запросов даёт 95%, а из 10 000 — 99.99%. Контекст объёма обязателен.

17. Alert как проверяемая гипотеза

Alert rule содержит выражение, длительность `for`, severity и annotations. Состояния:

1. inactive — условие ложно;
2. pending — условие истинно, но время `for` не прошло;
3. firing — условие сохраняется достаточно долго;
4. resolved — условие перестало выполняться.

Без `for` краткий всплеск немедленно активирует alert. Слишком длинный `for` скрывает реальный инцидент. Значение выбирают по стоимости ложного и пропущенного сигнала.

18. Alert должен вести к действию

Alert без владельца и runbook создаёт шум. Аннотация содержит краткое описание и ссылку на runbook. Runbook задаёт первые проверки, безопасные действия, критерий восстановления и требуемое evidence.

Не советуйте «перезапустить всё». Сначала фиксируются состояние, версия выпуска, target status и логи; затем выполняется минимальное обратимое действие.

19. Dashboard as code

Grafana datasource и dashboard provisioned из файлов. Это даёт review, одинаковый старт команд и восстановление на чистой машине. Ручные правки UI могут быть перезаписаны provisioning source и не принимаются без экспорта в Git.

Минимальные панели ЛР11: request rate, 5xx ratio, p95 по route, prediction outcomes, `up` и resolved model. Каждая панель содержит единицу, окно и осмысленный title.

20. Граница с мониторингом модели

ЛР11 измеряет service health и identity. Распределение prediction outcomes полезно, но без фактических labels не доказывает качество модели. Drift, delayed labels, coverage и оконные quality metrics рассматриваются в ЛР12.

Не называйте рост доли `delayed` деградацией модели без анализа входного распределения и фактических исходов.

21. Демонстрационный сценарий

1. Запустить app, Prometheus и Grafana через Compose.
2. Проверить targets и provisioned dashboard.
3. Создать 100 запросов с 10% ожидаемых 422.
4. Выполнить PromQL для rate, 5xx ratio и p95.
5. Найти JSON-log по request ID.
6. Показать model name/version из response, log и `model_info`.
7. Остановить app и наблюдать `up=0`, pending и firing.
8. Выполнить runbook и восстановить сервис.

22. Вопросы аудитории

1. Почему request ID опасен как label?
2. Почему 422 не включён в 5xx ratio?
3. Зачем `rate()` выполнять до `sum()`?
4. Почему p95 является оценкой?
5. Что доказывает `model_info`, а что не доказывает?
6. Когда alert должен иметь `for`?
7. Почему dashboard без provisioned source слаб как evidence?
8. Почему prediction distribution не равна model quality?

23. Связь с ЛР11 и ЛР12

В ЛР11 команда создаёт воспроизводимый observability stack, генерирует нагрузку, проверяет PromQL, JSON-log correlation, alert lifecycle и runbook. Каждый участник отвечает за отдельную metric/query/panel/alert/log/runbook часть и собственный сценарий.

В ЛР12 те же окна и release identity будут использованы для drift, delayed labels, coverage и оценки качества модели.

24. Официальные источники

- Prometheus instrumentation practices: https://prometheus.io/docs/practices/instrumentation/
- Prometheus query functions: https://prometheus.io/docs/prometheus/latest/querying/functions/
- Prometheus alerting rules: https://prometheus.io/docs/prometheus/latest/configuration/alerting_rules/
- Grafana provisioning: https://grafana.com/docs/grafana/latest/administration/provisioning/
- Grafana Docker installation: https://grafana.com/docs/grafana/latest/setup-grafana/installation/docker/
- Python Prometheus client releases: https://pypi.org/project/prometheus-client/